Cloud Administration Series

Senior Technical Architecture · Documentation Series

AWS Redis

Complete Technical Knowledge Base

Amazon ElastiCache (Valkey / Redis OSS) & Amazon MemoryDB

Architecture · Security · Performance · HA/DR · Operations

Drafted by	Praveen Madupu (PMSQLDBA)
Role	Senior SQL Server & Cloud DBA
Document	Technical Knowledge Base
Version	1.0
Date	July 2026
Classification	Internal / Confidential

Table of Contents

Amazon ElastiCache (Valkey / Redis OSS) & Amazon MemoryDB

1. Overview 3
2. Architecture 5
3. Components 8
4. Installation 10
5. Configuration 12
6. Prerequisites 14
7. Security Best Practices 15
8. Performance Tuning 17
9. Monitoring 18
10. Logging 20
11. Backup & Recovery 21
12. Disaster Recovery 22
13. High Availability 24
14. Troubleshooting 25
15. Common Errors 27
16. FAQs 28
17. Interview Questions 29
18. Real-world Use Cases 31
19. Best Practices 33
20. References 34

1. Overview

"AWS Redis" is not a single product. It is a family of fully managed, in-memory data services on AWS that speak the Redis wire protocol and API. In 2026 the family has two flagship services and one self-managed option:

- **Amazon ElastiCache** — the managed **cache** layer. It runs the **Valkey**, **Redis OSS**, and **Memcached** engines. Best for accelerating an existing database, storing sessions, leaderboards, rate limiters and pub/sub. Data is treated as **rebuildable**.
- **Amazon MemoryDB** — a **durable, primary in-memory database** that is Valkey- and Redis OSS-compatible. It persists every write to a **Multi-AZ transaction log** for 99.99% availability with no data loss, so it can be your system of record.
- **Self-managed Valkey / Redis on EC2** — you install and operate the engine yourself when you need full control of the OS, engine build, or a feature AWS has not yet exposed. You own HA, patching, and backups.

> **NOTE**
>
> In March 2024 Redis Inc. moved Redis 7.4+ to source-available licensing (SSPL/RSALv2; Redis 8.0 later adopted AGPLv3). In response, AWS and the community forked **Redis OSS 7.2** into **Valkey**, now stewarded by the Linux Foundation under the permissive BSD license. AWS positions **Valkey as the recommended engine** on ElastiCache and MemoryDB — it is API-compatible with Redis, and typically cheaper, faster, and more memory-efficient than the equivalent Redis OSS version.

1.1 Choosing the right AWS Redis service

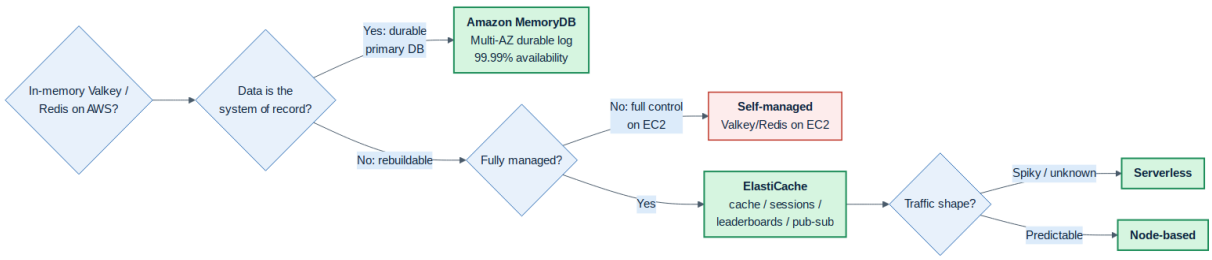


Figure 1.1 — Decision flow for selecting an AWS Redis service.

1.2 Service comparison at a glance

Dimension	ElastiCache (Valkey / Redis OSS)	MemoryDB (Valkey / Redis OSS)
Primary role	Cache / side data store	Durable primary database
Durability	In-memory; optional snapshots + AOF (single-AZ risk)	Multi-AZ transaction log — no data loss on failover
Read latency	Microseconds	Microseconds
Write latency	Microseconds	Single-digit milliseconds (durable commit)
Availability SLA	Up to 99.99% (Multi-AZ)	99.99%

Dimension	ElastiCache (Valkey / Redis OSS)	MemoryDB (Valkey / Redis OSS)
Deployment	Serverless or node-based	Node-based (Serverless-style scaling on newer versions)
Typical use	DB acceleration, sessions, queues	Microservices system-of-record, no separate cache+DB

1.3 Supported engines and versions (2026)

After you create a cluster you can **upgrade** the engine version, but you can never **downgrade** — you must recreate to move backward. Valkey is a drop-in replacement for Redis OSS 7 and supports cross-engine upgrades from Redis OSS.

Engine	Notable versions (2026)	Highlights
Valkey	7.2, 8.0, 8.1, 9.0, 9.1	Recommended. BSD-licensed. I/O multithreading, ~20–40% better memory efficiency, Bloom filters (8.1), native vector search (8.2+).
Redis OSS	6.2, 7.0, 7.1, 7.2	7.2 is the last fully open-source Redis. 6.2 added RBAC ACLs and native JSON.
Redis OSS (Extended Support)	4.0.x, 5.0.x	Standard support ended 2026-01-31; paid Extended Support runs to 2029-01-31. Upgrade off these.
Memcached	1.6.x	Multi-threaded simple cache; no persistence, replication, or rich data types (out of scope here).

BEST PRACTICE

For new workloads, start on **Valkey 8.1 or later**. You get the best price/performance, the longest support runway, and a permissive open-source license that avoids the AGPLv3 restrictions many enterprises forbid.

2. Architecture

Both services share the same logical building blocks — **clusters**, **shards**, and **nodes** — deployed inside your **VPC** across multiple **Availability Zones (AZs)**. They differ in how writes are made durable.

2.1 ElastiCache cluster architecture

An ElastiCache **cluster mode enabled** deployment partitions the keyspace into **16,384 hash slots** spread across **shards** (also called node groups). Each shard has one **primary** (accepts writes) and up to five **read replicas**. Placing replicas in different AZs from their primary is what delivers high availability.

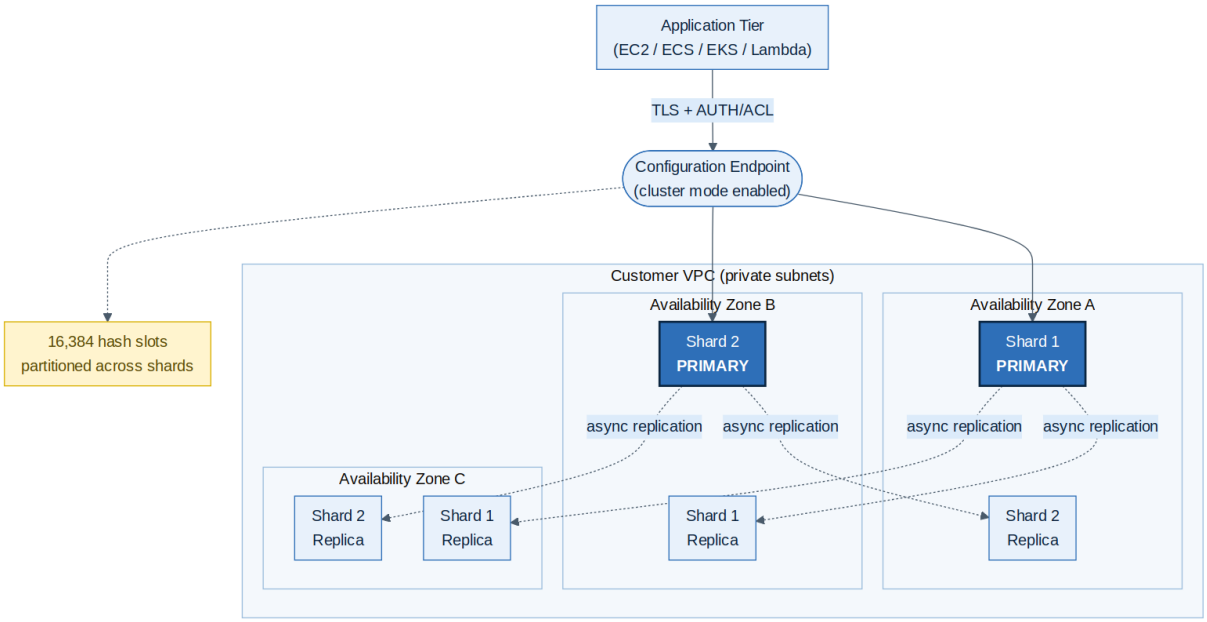


Figure 2.1 — ElastiCache (cluster mode enabled) across three Availability Zones.

Clients connect through a single **configuration endpoint**. A cluster-aware client issues `CLUSTER SLOTS` / `CLUSTER SHARDS` to learn the slot-to-node map, then talks **directly** to the node that owns each key. When a key's slot has moved, the node replies with a `-MOVED` or `-ASK` redirection.

> **NOTE**
>
> Replication in ElastiCache is **asynchronous**. On a primary failure a replica is promoted, but a few of the most recent writes that had not yet replicated can be lost. If you cannot tolerate that, use MemoryDB.

2.2 MemoryDB architecture (durable)

MemoryDB keeps the entire dataset in memory for microsecond reads, but it decouples **durability** from the engine. Every write on a primary is committed **synchronously to a distributed, Multi-AZ transaction log** before the client is acknowledged. Replicas apply only entries that are already durable in the log, so they never expose writes that could be rolled back.

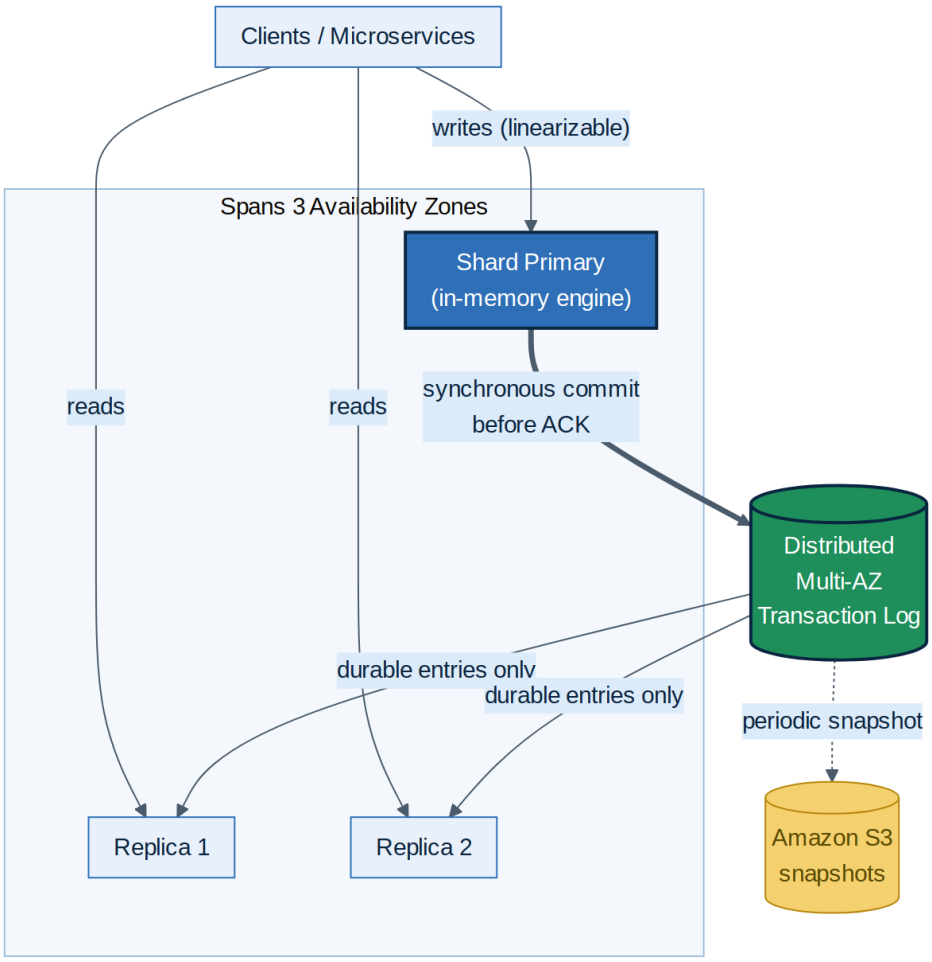


Figure 2.2 — MemoryDB decouples the in-memory engine from a Multi-AZ transaction log.

Property	ElastiCache	MemoryDB
Replication	Asynchronous	Via durable transaction log
Failover data loss	Possible (recent writes)	None
Consistency on primary	Strong until failover	Linearizable (durable-before-ack)
Snapshots	To AWS-managed S3	To AWS-managed S3, seeded from log

2.3 Cluster mode: disabled vs enabled

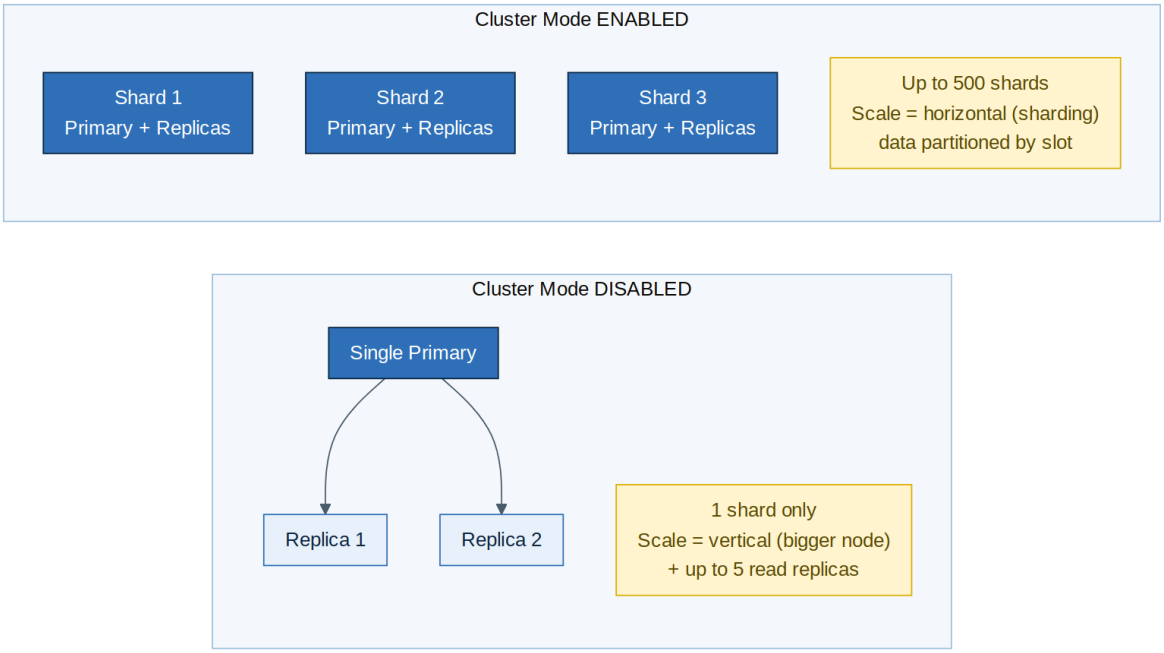


Figure 2.3 — Vertical scaling (cluster mode disabled) vs horizontal sharding (enabled).

Aspect	Cluster mode DISABLED	Cluster mode ENABLED
Shards	Exactly 1	Up to 500 (soft limit)
Scaling	Vertical (bigger node) + read replicas	Horizontal — add/remove shards, resharding
Keyspace	All keys on one primary	Keys spread across 16,384 slots
Multi-key ops	Any keys	Only keys in the same slot (use hash tags {..})
Best for	Small/medium datasets, simplicity	Large datasets, high write throughput

3. Components

Understanding the object model makes provisioning, troubleshooting, and IaC straightforward.

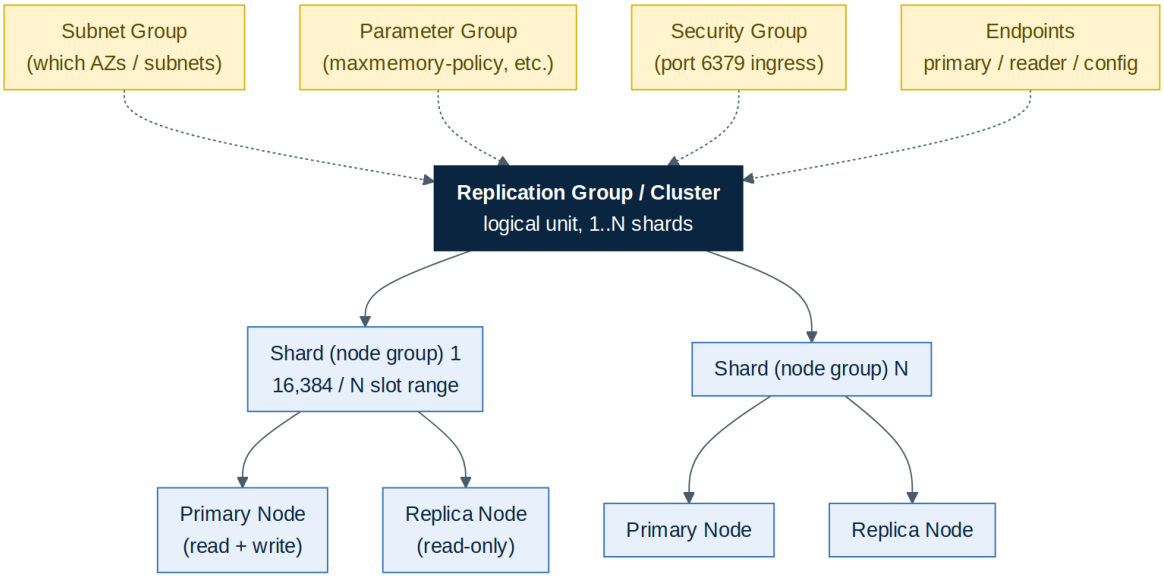


Figure 3.1 — AWS Redis object model and its supporting configuration objects.

Component	Description
Cluster / Replication group	The top-level managed object. In ElastiCache (cluster mode enabled) and MemoryDB it contains one or more shards; a "replication group" is the API term for a cluster with replicas.
Shard (node group)	A subset of the keyspace served by one primary and 0–5 replicas. Adding shards scales writes and memory horizontally.
Node	The smallest building block — one engine instance on an EC2 host of a chosen node type (e.g. cache.r7g.large). All nodes in a cluster share the same type.
Primary node	Accepts reads and writes for its shard. Exactly one per shard.
Replica node	Read-only copy that also provides HA — it can be promoted to primary on failover.
Subnet group	The set of VPC subnets (ideally one per AZ) in which nodes may be placed.
Parameter group	Engine configuration (maxmemory-policy, timeout, cluster settings). Applied cluster-wide.
Security group	Virtual firewall controlling inbound access (usually TCP 6379) to the nodes.
Endpoints	DNS names clients use: primary (writes), reader (load-balanced reads), and configuration (cluster discovery).
Snapshot (backup)	A point-in-time RDB copy stored in S3, used for restore, seeding, and migration.
Global Datastore	ElastiCache cross-region replication for low-latency global reads and regional DR.

3.1 Endpoint cheat-sheet

Endpoint	Use it for	Available when
Primary	All writes (cluster mode disabled)	Cluster mode disabled
Reader	Load-balanced reads across replicas	Replicas exist, cluster mode disabled
Configuration	Client auto-discovery of all shards	Cluster mode enabled / MemoryDB

4. Installation

"Installation" for a managed service means **provisioning** a cluster. You can use the Console, AWS CLI, CloudFormation, Terraform, or the SDK. The high-level flow is the same:

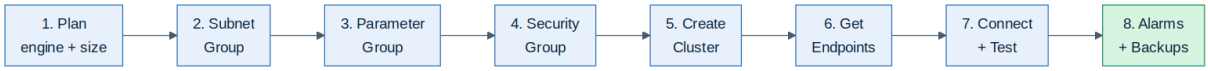


Figure 4.1 — End-to-end provisioning flow.

Step	What you do	Why
1 Plan	Pick engine (Valkey), version, node type, shard/replica count	Sizing drives cost and performance
2 Subnet group	Select private subnets across ≥ 2–3 AZs	Enables Multi-AZ placement
3 Parameter group	Clone default; set maxmemory-policy, timeouts	Engine tuning
4 Security group	Allow inbound 6379 from app tier only	Least-privilege network access
5 Create cluster	Enable Multi-AZ, auto-failover, encryption	HA + compliance
6 Endpoints	Copy configuration/primary/reader endpoints	Client connectivity
7 Connect + test	redis-cli --tls -a <token>	Verify reachability
8 Ops setup	Alarms, backups, maintenance window	Day-2 operations

4.1 AWS CLI — ElastiCache for Valkey (cluster mode enabled)

```
# 1) Subnet group across 3 AZs
aws elasticache create-cache-subnet-group \
  --cache-subnet-group-name vk-subnets \
  --cache-subnet-group-description "Valkey private subnets" \
  --subnet-ids subnet-0aaa subnet-0bbb subnet-0ccc

# 2) Custom parameter group (cluster-enabled family)
aws elasticache create-cache-parameter-group \
  --cache-parameter-group-name vk8-cme \
  --cache-parameter-group-family valkey8 \
  --description "Valkey 8 cluster-enabled tuned"
```

```
# 3) Create the replication group (3 shards, 2 replicas each = 9 nodes)
aws elasticache create-replication-group \
  --replication-group-id prod-valkey \
  --replication-group-description "Prod Valkey cache" \
  --engine valkey --engine-version 8.1 \
  --cache-node-type cache.r7g.large \
  --num-node-groups 3 --replicas-per-node-group 2 \
  --cache-subnet-group-name vk-subnets \
  --cache-parameter-group-name vk8-cme \
  --security-group-ids sg-0redis \
  --automatic-failover-enabled --multi-az-enabled \
  --transit-encryption-enabled --at-rest-encryption-enabled \
  --snapshot-retention-limit 7 --snapshot-window 03:00-04:00 \
  --preferred-maintenance-window sun:05:00-sun:06:00
```

4.2 AWS CLI — Amazon MemoryDB

```
aws memorydb create-subnet-group \
  --subnet-group-name mdb-subnets \
  --subnet-ids subnet-0aaa subnet-0bbb subnet-0ccc

aws memorydb create-cluster \
  --cluster-name prod-memorydb \
  --engine valkey --engine-version 7.2 \
  --node-type db.r7g.large \
  --num-shards 2 --num-replicas-per-shard 2 \
  --subnet-group-name mdb-subnets \
  --security-group-ids sg-0redis \
  --acl-name my-acl \
  --tls-enabled \
  --snapshot-retention-limit 7
```

4.3 Infrastructure as Code (Terraform, ElastiCache)

```
resource "aws_elasticache_replication_group" "valkey" {
  replication_group_id = "prod-valkey"
  description          = "Prod Valkey cache"
  engine               = "valkey"
  engine_version       = "8.1"
  node_type            = "cache.r7g.large"
  num_node_groups         = 3
  replicas_per_node_group = 2
  automatic_failover_enabled = true
  multi_az_enabled           = true
  transit_encryption_enabled = true
  at_rest_encryption_enabled = true
  subnet_group_name  = aws_elasticache_subnet_group.vk.name
  security_group_ids = [aws_security_group.redis.id]
  snapshot_retention_limit = 7
}
```

TIP

Codify clusters in Terraform/CloudFormation from day one. It makes environments reproducible across dev/test/prod, supports peer review, and gives you a clean rollback path — the same discipline you apply to database change control.

5. Configuration

Engine behavior is controlled by **parameter groups**. You cannot edit the default group — clone it, change parameters, and attach the custom group. Most parameters are **dynamic** (apply without reboot); a few are **static** and require a reboot or apply during the maintenance window.

5.1 High-impact parameters

Parameter	Purpose	Typical value
`maxmemory-policy`	Eviction policy when memory is full	`volatile-lru` (cache) / `noeviction` (MemoryDB primary)
`maxmemory-percent`	Reserved memory managed by ElastiCache	Leave AWS default (reserved-memory-percent)
`timeout`	Close idle client connections (seconds)	300 (0 = never)
`tcp-keepalive`	Detect dead peers	300
`cluster-enabled`	Sharded (cluster) mode	yes (via cluster-enabled family)
`activedefrag`	Active memory defragmentation	yes for long-lived, fragmented caches
`slowlog-log-slower-than`	Threshold (µs) to record slow commands	10000 (10 ms)
`notify-keyspace-events`	Keyspace notifications for pub/sub	Enable only if used (costs CPU)

5.2 Eviction policies

Policy	Behavior
`noeviction`	Reject writes when full (returns OOM). Correct for a durable primary.
`allkeys-lru` / `allkeys-lfu`	Evict least-recently / least-frequently used across all keys. Good general cache policy.
`volatile-lru` / `volatile-lfu`	Evict only keys that have a TTL set. Protects keys you never want dropped.
`allkeys-random` / `volatile-random`	Evict at random. Rarely optimal.
`volatile-ttl`	Evict keys with the nearest expiry first.

5.3 Applying parameter changes

```
# Set an eviction policy on a custom parameter group
aws elasticache modify-cache-parameter-group \
  --cache-parameter-group-name vk8-cme \
  --parameter-name-values \
      "ParameterName=maxmemory-policy,ParameterValue=allkeys-lru" \
      "ParameterName=timeout,ParameterValue=300"
```

```
# Verify a running cluster's effective config from redis-cli
redis-cli --tls -h <endpoint> -a <token> CONFIG GET maxmemory-policy
```

WARNING

Some parameters are **static** and only take effect after the node reboots. Schedule the reboot in your maintenance window and expect a brief failover on the affected shard.

6. Prerequisites

Have these in place before you provision — they prevent the most common "cluster created but nothing connects" issues.

Prerequisite	Detail
AWS account & IAM	IAM principal with elasticache:* / memorydb:* for the control plane; least-privilege in prod.
VPC & subnets	A VPC with private subnets in at least 2 (ideally 3) AZs for Multi-AZ. No public exposure.
Subnet group	Registers those subnets with the service.
Security group	Inbound TCP 6379 (or your chosen port) from the application security group only.
KMS key	For encryption at rest — AWS-managed or a customer-managed CMK for tighter control/audit.
AUTH / ACL / IAM auth	Decide the authentication model and store secrets in AWS Secrets Manager.
Client library	A cluster-aware client (redis-py, Lettuce/Jedis, go-redis, StackExchange.Redis, node-redis) with TLS support.
Sizing plan	Estimate working-set memory + overhead, peak ops/sec, and network throughput to pick node type and shard count.

BEST PRACTICE

Size memory for the **working set plus overhead**: leave headroom (AWS reserves memory for replication buffers and background saves — commonly ~25%). Running a cache above ~90% `DatabaseMemoryUsagePercentage` triggers evictions and latency spikes.

7. Security Best Practices

Security is layered. No single control is sufficient — combine network isolation, encryption, authentication, and governance (defense in depth).



Figure 7.1 — Defense-in-depth layers for AWS Redis.

7.1 Controls

Layer	Control	How
Network	VPC isolation	Private subnets, no public IP; SG allows only the app tier; consider VPC endpoints.
Transport	TLS in transit	Enable `transit-encryption`; connect with `--tls`; enforce TLS-only.
AuthN	AUTH token / ACL / IAM	Redis AUTH token, or Valkey/Redis 6+ ACL users (RBAC), or IAM authentication (no static secret).
AuthZ	Least-privilege ACLs	Grant per-user command and key-pattern permissions; avoid the default user with full rights.
At rest	KMS encryption	Enable at-rest encryption with a customer-managed CMK; Graviton2 adds always-on DRAM encryption.
Governance	Audit & rotation	CloudTrail for API calls; rotate AUTH tokens/secrets via Secrets Manager; tag for ownership.

7.2 Authenticate with IAM (no stored password)

```
# ElastiCache/MemoryDB IAM auth: the "password" is a short-lived
# IAM auth token generated at connect time, so no static secret is stored.
TOKEN=$(aws elasticache generate-iam-auth-token \
  --cluster-name prod-valkey --user-id app-svc --region us-east-1)

redis-cli --tls -h <endpoint> --user app-svc --pass "$TOKEN" PING
```

7.3 Role-based ACL user

```
# Read-only user limited to the "app:" keyspace
aws elasticache create-user \
  --user-id reporting-ro --user-name reporting \
  --engine redis --passwords "REDACTED-STRONG-TOKEN" \
  --access-string "on ~app:* +@read -@dangerous"
```

BEST PRACTICE

For **SOX / PCI-DSS / HIPAA** environments (relevant to financial-services engagements): enable encryption in transit and at rest with a **customer-managed CMK**, disable the default ACL user, require **IAM or rotated ACL**

> authentication, keep clusters in **private subnets with no internet route**, and stream **CloudTrail + engine/slow logs** to a monitored, immutable location for your evidence trail.

8. Performance Tuning

Redis is fast by default; most latency problems come from **hot keys, slow commands, undersized nodes, or chatty clients**. Tune in this order: data model → client behavior → topology → node size.

Technique	What it does	Watch out for
Connection pooling	Reuse connections; avoid TLS handshake per request	Pool too small → queueing; too large → node connection limits
Pipelining / MGET/MSET	Batch commands to cut round trips	Huge pipelines block the event loop
Avoid O(N) commands	No `KEYS *`, big `SMEMBERS`, unbounded `SORT`	Use `SCAN`, secondary indexes, pagination
Right-size nodes	More vCPU/memory/network headroom	Redis is single-threaded per shard for command exec
Cluster mode / sharding	Parallelize across shard primaries	Multi-key ops must share a slot (hash tags)
Read replicas	Offload reads from primary	Replica reads are eventually consistent
Enhanced I/O multiplexing	More throughput at high connection counts	Biggest win on Valkey/Redis 7+ with many clients
Data tiering (r6gd)	Warm data in memory, cold on SSD — larger sets cheaper	Adds latency for SSD-resident keys
Client-side caching	Cache hot values in the app (tracking)	Invalidation complexity
TTLs + eviction policy	Keep the working set bounded	Wrong policy evicts hot keys

WARNING

A single expensive command (for example `KEYS *` on a large keyspace) blocks the engine for **every** client on that shard, because command execution is single-threaded. Find offenders with the slow log and replace them with `SCAN` or a proper index.

8.1 Find slow commands

```
redis-cli --tls -h <endpoint> -a <token> SLOWLOG GET 10
redis-cli --tls -h <endpoint> -a <token> SLOWLOG RESET
# INFO sections for a quick health read
redis-cli --tls -h <endpoint> -a <token> INFO stats
redis-cli --tls -h <endpoint> -a <token> INFO commandstats
```

9. Monitoring

AWS Redis publishes host- and engine-level metrics to **Amazon CloudWatch**. Build alarms on the leading indicators of trouble and route them to your on-call channel.

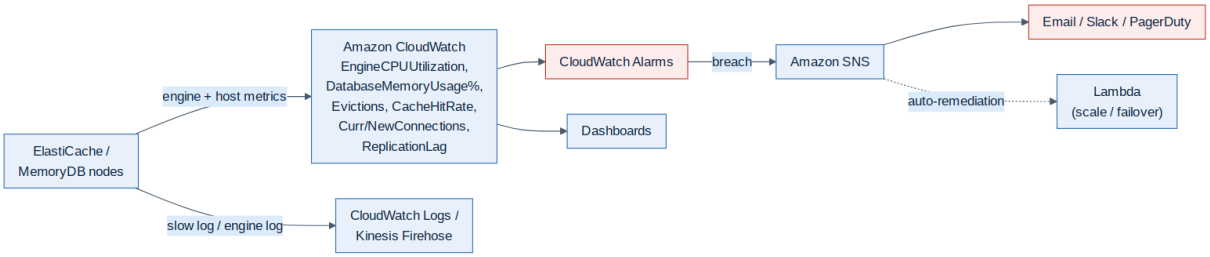


Figure 9.1 — Metrics, alarms, and notification flow.

9.1 Key metrics and suggested thresholds

Metric	Meaning	Alarm guidance
`EngineCPUUtilization`	CPU used by the Redis/Valkey engine thread	> 90% sustained → scale up / shard
`CPUUtilization`	Host CPU (all cores)	Context for background tasks
`DatabaseMemoryUsagePercentage`	Memory used vs available	> 90% → evictions imminent
`Evictions`	Keys evicted due to maxmemory	Any sustained rise on a primary
`CacheHitRate`	Hits / (hits + misses)	Falling rate = cache not effective
`CurrConnections` / `NewConnections`	Open and new client connections	Near node max → pooling/leak issue
`ReplicationLag`	Seconds a replica is behind primary	High → stale reads / failover risk
`SwapUsage`	OS swap in use	> 50 MB → memory pressure
`NetworkBandwidthOutAllowanceExceeded`	Network cap hit	Any nonzero → right-size node
`ReplicationBytes` / `SaveInProgress`	Replication & snapshot activity	Correlate with latency spikes

9.2 Create an alarm (CLI)

```
aws cloudwatch put-metric-alarm \
  --alarm-name valkey-mem-90 \
  --namespace AWS/ElastiCache \
  --metric-name DatabaseMemoryUsagePercentage \
  --dimensions Name=ReplicationGroupId,Value=prod-valkey \
  --statistic Average --period 60 --evaluation-periods 5 \
  --threshold 90 --comparison-operator GreaterThanThreshold \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:oncall
```

> **TIP**
>
> Alarm on **trends and rates**, not just instantaneous values. A slow, steady climb in `Evictions` or `DatabaseMemoryUsagePercentage` is your early warning to scale before customers feel it.

10. Logging

There are three distinct log sources. Do not confuse control-plane audit logs (CloudTrail) with engine logs.

Log	Contains	Destination
Slow log	Commands exceeding `slowlog-log-slower-than`	CloudWatch Logs or Kinesis Data Firehose
Engine log	Engine events, failovers, warnings	CloudWatch Logs or Firehose
CloudTrail	Control-plane API calls (create/modify/delete)	CloudTrail → S3 / CloudWatch

10.1 Enable engine & slow logs (JSON to CloudWatch)

```
aws elasticache modify-replication-group \
  --replication-group-id prod-valkey --apply-immediately \
  --log-delivery-configurations \
    "LogType=slow-log,DestinationType=cloudwatch-logs,\
DestinationDetails={CloudWatchLogsDetails={LogGroup=/elasticache/prod/slow}},\
LogFormat=json,Enabled=true" \
    "LogType=engine-log,DestinationType=cloudwatch-logs,\
DestinationDetails={CloudWatchLogsDetails={LogGroup=/elasticache/prod/engine}},\
LogFormat=json,Enabled=true"
```

BEST PRACTICE

Ship logs to a **centralized, access-controlled** log group with retention that meets your compliance mandate, and build a CloudWatch Logs Insights query or metric filter for `-MOVED`, failover, and OOM events so incidents surface automatically.

11. Backup & Recovery

Snapshots are point-in-time **RDB** copies stored by AWS in S3. Use them for restore, cloning to a new topology, and migration. Snapshots are taken off the primary's workload (typically from a replica) to minimize impact.



Figure 11.1 — Snapshot, export, and restore/seed flow.

Concept	Detail
Automatic backups	Daily during the backup window; set `snapshot-retention-limit` (1–35 days).
Manual snapshots	On demand; retained until you delete them — use before risky changes.
Export to S3	Copy a snapshot to **your** bucket as a portable `.rdb` for archival or cross-account use.
Restore / seed	Create a new cluster from a snapshot, optionally with a **different node type or shard count**.
MemoryDB	Durable by design; snapshots complement the transaction log for long-term retention & cloning.

```
# Manual snapshot before a major change
aws elasticache create-snapshot \
  --replication-group-id prod-valkey \
  --snapshot-name prod-valkey-pre-upgrade

# Restore into a new, resharded cluster
aws elasticache create-replication-group \
  --replication-group-id prod-valkey-restored \
  --replication-group-description "Restore test" \
  --engine valkey --snapshot-name prod-valkey-pre-upgrade \
  --num-node-groups 4 --replicas-per-node-group 1 \
  --cache-node-type cache.r7g.large
```

WARNING

A backup you have never restored is a hope, not a plan. **Rehearse restores** on a schedule and record the actual time-to-recover — that number is your real RTO.

12. Disaster Recovery

DR protects against the loss of an entire **Region**. ElastiCache provides **Global Datastore** — asynchronous cross-region replication with one primary (read/write) region and up to two secondary (read-only) regions. MemoryDB offers a Multi-Region capability with similar goals.

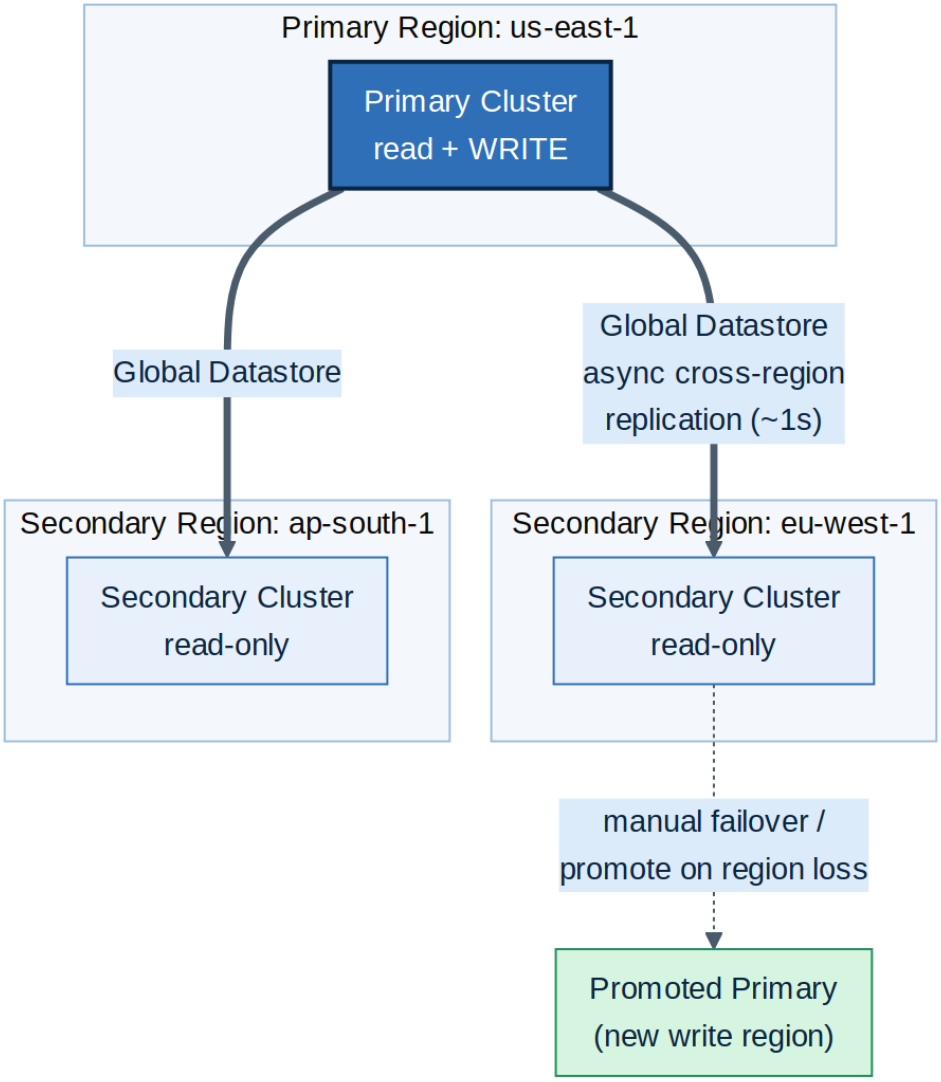


Figure 12.1 — Global Datastore: one writable region replicating to read-only secondaries.

Term	Meaning	Typical AWS Redis figure
RPO (Recovery Point Objective)	Acceptable data loss	Near-zero to ~1s (async cross-region lag)
RTO (Recovery Time Objective)	Time to restore service	Minutes (manual promote of a secondary)
Cross-region replication	Keeps a warm copy elsewhere	Global Datastore / MemoryDB Multi-Region

12.1 Promote a secondary region

```
# During a regional event, promote the secondary to a standalone
# writable cluster and repoint the application (or Route 53).
aws elasticache failover-global-replication-group \
  --global-replication-group-id my-global-store \
  --primary-region eu-west-1 \
  --primary-replication-group-id prod-valkey-euw1
```

BEST PRACTICE

Document runbooks for **failover and fail-back**, automate DNS repointing (Route 53), and test a region-loss game day at least twice a year. DR that has never been exercised will not work when you need it.

13. High Availability

HA protects against the loss of a **node or an AZ within a Region**. It is delivered by placing replicas in different AZs and enabling **automatic failover**.

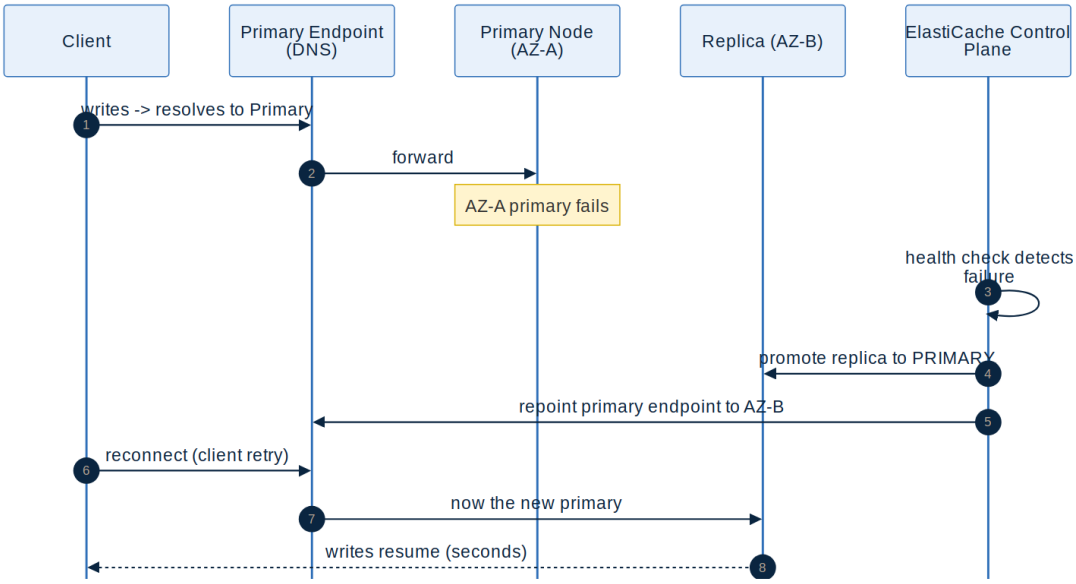


Figure 13.1 — Automatic Multi-AZ failover sequence.

- **Multi-AZ** — replicas live in different AZs from their primary. Enable it plus **automatic failover** for production.
- **Automatic failover** — on primary failure the service promotes the most up-to-date replica and repoints the endpoint DNS, usually within seconds.
- **Client behavior** — cluster-aware clients handle the DNS/topology change and retry. Configure sensible connect/command timeouts and retry-with-backoff.
- **MemoryDB** — failover is **lossless** because promoted replicas apply only durable log entries; ElastiCache failover may drop the last few unreplicated writes.

Design choice	HA impact
At least 1 replica per shard	Required for automatic failover
Replicas spread across AZs	Survive a full AZ outage
Automatic failover enabled	No manual intervention on node loss
Client retries + timeouts	Ride through the brief failover window

14. Troubleshooting

Start from the **symptom**, confirm with a metric, then act. The tree below covers the majority of production incidents.

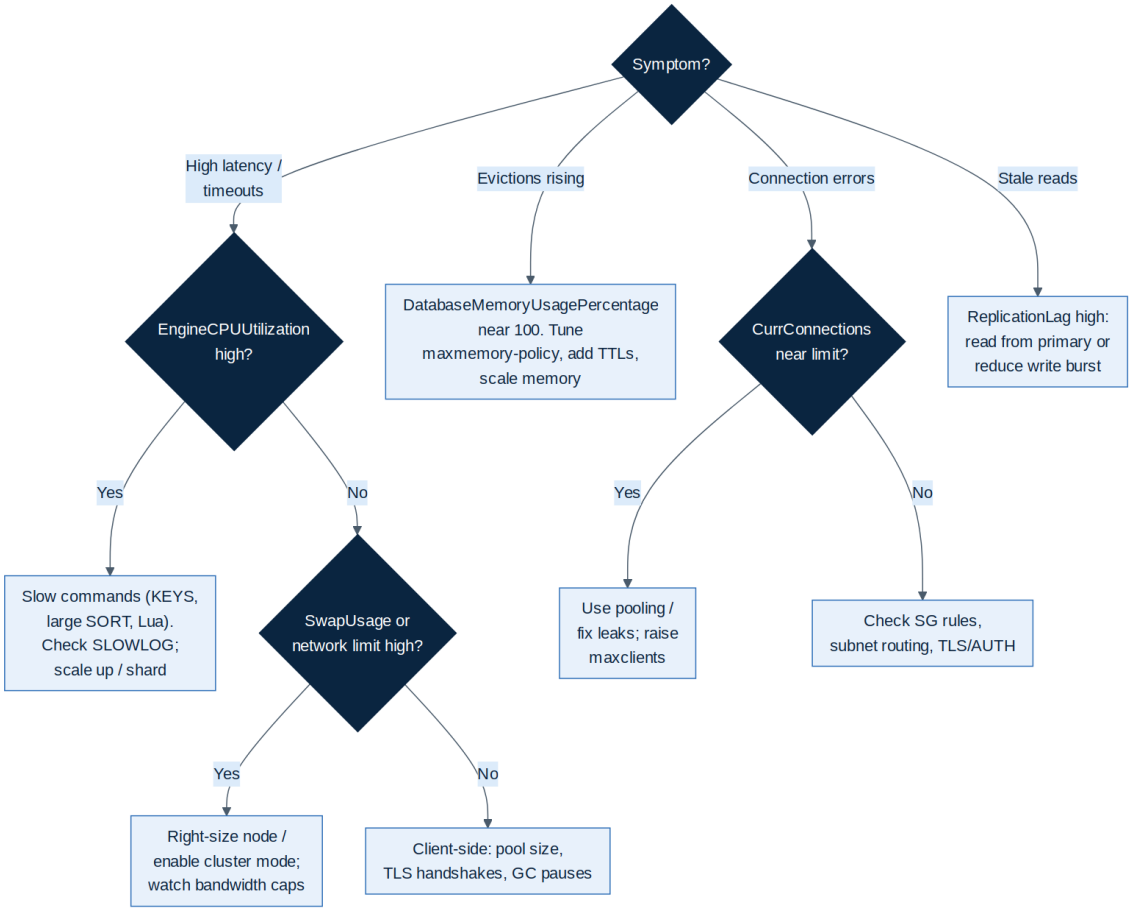


Figure 14.1 — Symptom-driven troubleshooting decision tree.

14.1 Playbook

Symptom	First metric to check	Likely fix
High latency / timeouts	`EngineCPUUtilization`, `SLOWLOG`	Remove O(N) commands; scale up; add shards
Evictions climbing	`DatabaseMemoryUsagePercentage`	Add TTLs; tune eviction policy; add memory/shards
Connection errors	`CurrConnections`, SG rules	Pool connections; fix leaks; verify SG & TLS/AUTH
Stale reads	`ReplicationLag`	Read from primary or reduce write bursts
Intermittent -MOVED errors	Client cluster-awareness	Use a cluster-mode client; refresh slot map
OOM on writes	`maxmemory-policy`	Free memory / change policy / scale (durable = noeviction expected)

15. Common Errors

Reference for the error strings clients most often surface, with cause and resolution.

Error	Cause	Resolution
`MOVED <slot> <host:port>`	Key's slot is on another shard (cluster mode)	Use a cluster-aware client; it follows the redirect automatically
`ASK <slot> <host:port>`	Slot is migrating during resharding	Client should `ASKING` then retry on the target node
`CLUSTERDOWN Hash slot not served`	Not all slots are covered (node down / mid-reshard)	Wait for failover/heal; check node health
`OOM command not allowed when used memory > maxmemory`	Memory full with `noeviction`	Free memory, add TTLs, scale, or change policy (if a cache)
`MISCONF Redis is configured to save RDB snapshots...`	Background save failing (self-managed)	Managed services handle this; on EC2 fix disk/permissions
`NOAUTH Authentication required`	AUTH enabled but no credential sent	Send AUTH token / user+password / IAM token
`WRONGPASS invalid username-password pair`	Bad AUTH token or ACL credentials	Rotate/verify token; check ACL user
`ERR max number of clients reached`	Connection limit hit	Pool connections; fix leaks; raise maxclients / scale
`Connection timed out`	SG/NACL/route or wrong endpoint/TLS	Verify SG inbound 6379, subnet routing, `--tls`, endpoint
`READONLY You can't write against a read only replica`	Write sent to a replica/reader endpoint	Send writes to the primary/configuration endpoint

16. FAQs

Q1. Is "AWS Redis" the same as Redis?

A. It is Redis-compatible. AWS runs the **Valkey** and **Redis OSS** engines on ElastiCache and MemoryDB. Valkey is the community BSD-licensed fork of Redis OSS 7.2 and is a drop-in replacement.

Q2. Should I pick Valkey or Redis OSS?

A. Valkey for new work — it is API-compatible, generally faster and more memory-efficient, cheaper, has a permissive license, and the longest support runway.

Q3. ElastiCache vs MemoryDB — which one?

A. ElastiCache is a **cache** (data is rebuildable). MemoryDB is a **durable primary database** (data survives failures with no loss). If losing recent writes on failover is unacceptable, use MemoryDB.

Q4. Serverless or node-based?

A. Serverless removes capacity planning and scales instantly — ideal for spiky/unknown traffic. Node-based gives fine control and can be cheaper for steady, predictable load.

Q5. Does failover lose data?

A. On ElastiCache, possibly — replication is async, so the last few unreplicated writes can be lost. On MemoryDB, no — failover uses the durable log.

Q6. How do I connect securely?

A. Over TLS from within the VPC, authenticating with an AUTH token, ACL user, or (best) IAM authentication so no static secret is stored.

Q7. Can I run multi-key transactions?

A. Yes, if all keys map to the same slot. In cluster mode use **hash tags** — e.g. `{user123}:profile` and `{user123}:cart` share a slot.

Q8. How are backups stored?

A. As point-in-time RDB snapshots in AWS-managed S3; you can export a portable copy to your own bucket.

Q9. What is the availability SLA?

A. Up to 99.99% with a Multi-AZ configuration (MemoryDB and Multi-AZ ElastiCache).

Q10. How do I migrate from self-managed Redis?

A. Seed a new cluster from an exported `.rdb` snapshot via `--snapshot-arns`, or use online migration tooling; cut over by repointing the endpoint.

17. Interview Questions

Grouped by theme, with model answers — useful for cloud/data-engineering interview preparation.

Fundamentals

What is Amazon ElastiCache and when would you use it?
A managed in-memory cache (Valkey/Redis OSS/Memcached) used to accelerate reads, store sessions, and offload a backing database. Use it when repeated reads are expensive and the data can be rebuilt from the source of truth.

How does MemoryDB differ from ElastiCache?
MemoryDB is a **durable** primary database: writes commit to a Multi-AZ transaction log before ack, so there is no data loss on failover. ElastiCache is a cache with async replication.

Explain hash slots.
Redis cluster mode partitions the keyspace into 16,384 slots; each shard owns a slot range. `CRC16(key) mod 16384` picks the slot, so a cluster-aware client routes each key to the right node.

Architecture & HA

How does automatic failover work?
With Multi-AZ and a replica present, the service detects primary failure, promotes the most current replica, and repoints the endpoint DNS — typically within seconds.

Cluster mode enabled vs disabled?
Disabled = single shard, scale vertically + up to 5 read replicas. Enabled = up to 500 shards, scale horizontally, keys spread by slot; multi-key ops need a shared slot.

How do you achieve cross-region DR?
ElastiCache Global Datastore (or MemoryDB Multi-Region): async replication to read-only secondary regions; promote a secondary on regional loss.

Performance

Why can one command slow the whole shard?
Command execution per shard is single-threaded, so an O(N) command like `KEYS *` blocks all other clients on that shard until it completes.

How do you diagnose latency?
Check `EngineCPUUtilization`, `SLOWLOG`, `INFO commandstats`, network allowance metrics, and `ReplicationLag`; then remove slow commands, scale, or shard.

What is data tiering?
On r6gd nodes, warm data stays in memory and cold data moves to local SSD, letting you hold larger datasets more cheaply at some latency cost for SSD-resident keys.

Security & Ops

How do you secure an AWS Redis cluster?

Private subnets + tight SG, TLS in transit, KMS at rest, IAM/ACL authentication with least privilege, CloudTrail auditing, and secret rotation via Secrets Manager.

How do backups and restore work?

Automatic daily + manual RDB snapshots to S3; restore into a new cluster, optionally resharded, and export a portable copy to your own bucket.

Which caching pattern do you use and why?

Cache-aside (lazy loading) is the default: the app checks the cache, falls back to the DB on a miss, and populates the cache with a TTL. Add write-through when you need the cache always fresh.

18. Real-world Use Cases

The dominant pattern is **cache-aside (lazy loading)** — check the cache, fall back to the database on a miss, and repopulate with a TTL.

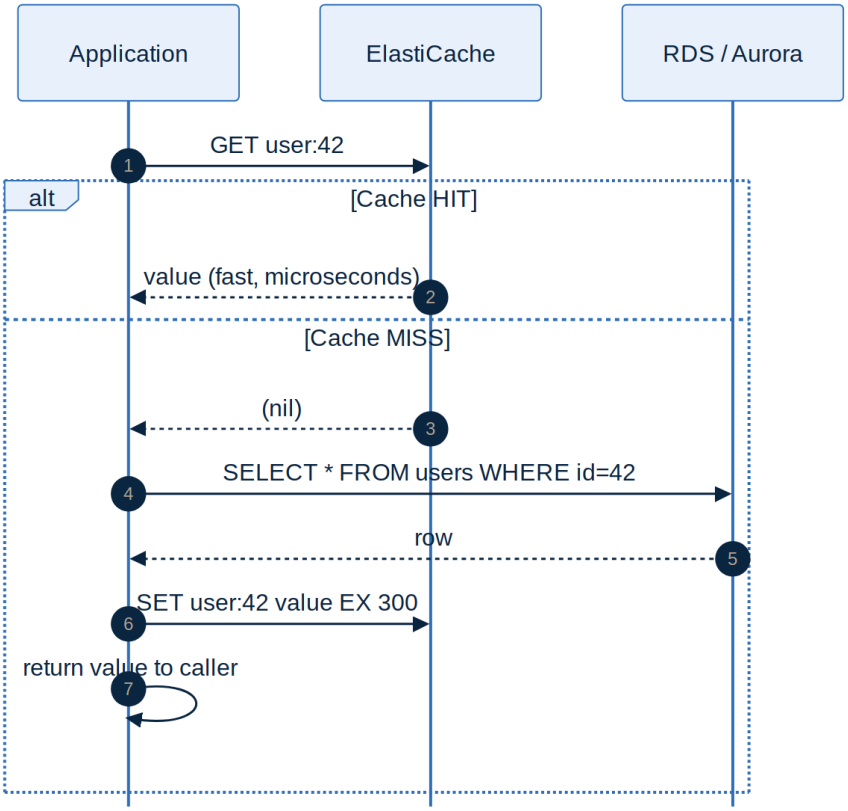
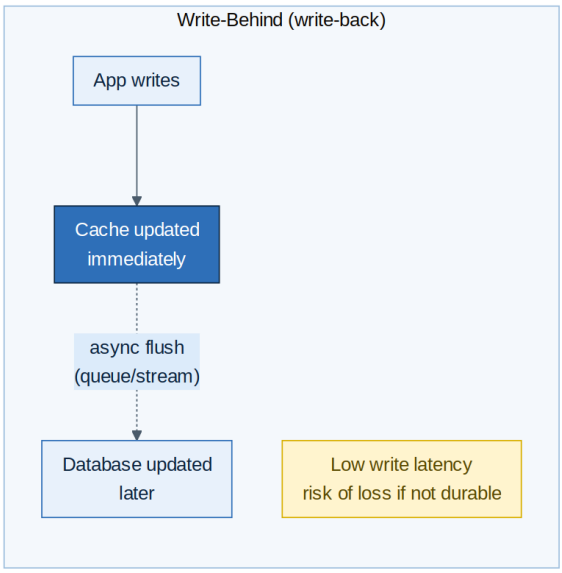


Figure 18.1 — Cache-aside (lazy loading) sequence.

When you need the cache to always mirror the database, use **write-through** (update cache and DB together) or **write-behind** (update cache now, flush to DB asynchronously).



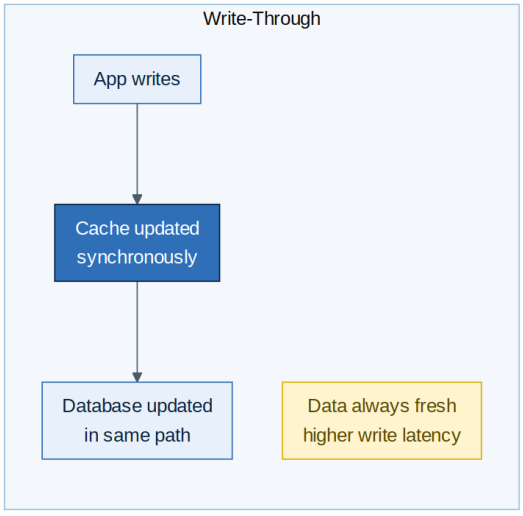


Figure 18.2 — Write-through vs write-behind.

18.1 Common workloads

Use case	How AWS Redis helps	Structures / features
Database caching	Absorb hot reads, cut DB load & latency	Strings/hashes, TTL, cache-aside
Session store	Fast, shared session state across app nodes	Hashes + TTL
Leaderboards / gaming	Real-time ranking at scale	Sorted sets (ZADD/ZRANGE)
Rate limiting	Throttle APIs per user/IP	INCR + EXPIRE, sliding windows
Pub/Sub & queues	Fan-out messaging, work queues	Pub/Sub, Streams, Lists
Geospatial	Nearby search (ride-share, delivery)	GEOADD / GEOSEARCH
ML feature store	Low-latency online features	Hashes, MemoryDB for durability
Vector / semantic search	RAG, recommendations, semantic caching	Vector search (MemoryDB, Valkey 8.2+)

18.2 Rate limiter example

```
-- Atomic fixed-window limiter (Lua): max N requests per window
-- KEYS[1] = rate:{user}:{window}, ARGV[1] = limit, ARGV[2] = ttl
local c = redis.call('INCR', KEYS[1])
if c == 1 then redis.call('EXPIRE', KEYS[1], ARGV[2]) end
if c > tonumber(ARGV[1]) then return 0 else return 1 end
```

19. Best Practices

19.1 Design

- Default to **Valkey (latest)**; choose **MemoryDB** when data must be durable, **ElastiCache** when it is a cache.
- Always set **TTLs**; keep the working set bounded and pick an eviction policy that matches your intent.
- Use **hash tags** to co-locate related keys when you need multi-key operations in cluster mode.
- Model data with the right structure (sorted sets for ranking, hashes for objects) rather than serializing blobs.

19.2 Availability & operations

- Enable **Multi-AZ + automatic failover**; run at least one replica per shard across AZs.
- Turn on **automatic backups**, rehearse restores, and record real RTO/RPO.
- Codify everything in **IaC**; change clusters through reviewed pipelines.
- Right-size on evidence: watch CPU, memory %, evictions, connections, and network allowance metrics.

19.3 Security & cost

- Private subnets, tight security groups, **TLS + KMS**, and **IAM/ACL least privilege**; audit with CloudTrail.
- Rotate AUTH tokens/secrets via **Secrets Manager**; disable the default ACL user.
- Use **reserved nodes** or **Serverless** to match spend to the traffic pattern; consider **data tiering** for large, cold-heavy datasets.
- Alarm on **trends** (rising evictions/memory) to scale before users are affected.

> **BEST PRACTICE**
>
> The single highest-leverage habit: **set a TTL on every cache key and alarm on evictions and memory percentage.** Most production Redis incidents trace back to unbounded growth and the wrong eviction policy.

20. References

Official AWS documentation and primary sources (verify against the console for the latest limits and versions):

- Amazon ElastiCache — User Guide — https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/
- ElastiCache — Engine versions & upgrading — https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/engine-versions.html
- ElastiCache — Choosing an engine (Valkey / Redis OSS / Memcached) — https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/SelectEngine.html
- Redis OSS vs. Valkey (AWS) — https://aws.amazon.com/elasticache/redis/
- Amazon MemoryDB — What is MemoryDB — https://docs.aws.amazon.com/memorydb/latest/devguide/what-is-memorydb.html
- Amazon MemoryDB — Features & FAQs — https://aws.amazon.com/memorydb/features/
- ElastiCache — Global Datastore (cross-region) — https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/Redis-Global-Datastore.html
- ElastiCache — Monitoring with CloudWatch metrics — https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/CacheMetrics.html
- ElastiCache — Backup and restore — https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/backups.html
- ElastiCache — In-transit & at-rest encryption, auth — https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/encryption.html
- Valkey project (Linux Foundation) — https://valkey.io/
- Redis commands reference — https://redis.io/commands/

End of document